



GitHub Trending Top 10

每日热门开源项目深度解读

2026-10-09

今日榜单

1	morluto/rea	TypeScript	Reverse engineer anything with agents, from app behavior down to native binaries...	4天
2	boykopovar/AnyPS5	C++	Tool for automatic PS5 executables porting to Linux and Windows	5天
3	mattpocock/skills	Shell	Skills for Real Engineers. Straight from my .agents directory.	4天
4	cathrynlavery/diagram-design	HTML	Editorial diagram design for Claude Code, Codex, GitHub Copilot, Factory Droid, ...	3天
5	alibaba/open-code-review	Go	Secure, fast, efficient, battle-tested at Alibaba's scale. Hybrid architecture c...	NEW
6	anthropics/knowledge-work-plugins	Python	Open source repository of plugins primarily intended for knowledge workers to us...	2天
7	BerriAI/litellm	Python	The fastest, lightest AI Gateway. Rust core with Python SDK. Call 100+ LLM APIs in...	NEW
8	addyosmani/agent-skills	JavaScript	Production-grade engineering skills for AI coding agents.	NEW
9	storytold/artcraft	Rust	ArtCraft is an intentional crafting engine for artists, designers, and filmmaker...	2天
10	Robbyant/lingbot-map	Python	[ECCV 2026 Best Paper Award Candidate] LingBot-Map: Geometric Context Transforme...	NEW

#1

TypeScript

连续 4 天

rea

Reverse engineer anything with agents, from app behavior down to native binaries.

Stars **39,358** Forks **6,025** Today **+15,335**

<https://github.com/morluto/rea>

项目简介：REA 是一个基于 MCP（Model Context Protocol）的逆向工程工具集，让 AI 智能体能够分析原生二进制、JavaScript/Electron 应用、.NET 程序集以及网站。它解决的核心问题是：当你看到一个心仪的产品功能时，无需源码就能理解其实现原理，并让 AI 帮你复刻到自己的项目中。

核心功能：一是通过 MCP 协议将逆向分析能力接入 Claude Code、Cursor 等主流 AI 智能体；二是支持多层级分析——从应用行为、JavaScript/Electron 代码到原生二进制（借助 Hopper/Ghidra）；三是提供终端 CLI 工具（rea-agents），可独立运行分析任务；四是所有分析本地执行，并输出带证据和局限性的结论；五是支持静态 JavaScript 分析，无需额外引擎即可使用。

适用场景：适合想研究竞品功能实现的产品开发者、需要理解第三方应用行为的安全研究人员，以及希望从现有软件中学习架构设计的工程师。典型用法是让 AI 智能体调查某个 App 的特定功能（如搜索机制），获取证据链后在自己的项目中重建类似能力。

技术亮点：项目深度整合 MCP 协议，把逆向工程这种专业操作封装成 AI 智能体可直接调用的工具链，大幅降低了使用门槛。它同时支持动态桥接（如 Hopper 内启动分析桥）和静态分析，并强调"证据可追溯"——每个结论都附带分析依据和局限性说明，这在逆向工具中较为少见。此外，一键 setup 能自动配置多个主流智能体并备份原有配置，工程化程度高。

上手建议：最快的方式是运行 `npx rea-agents setup`，选择你的 AI 智能体并批准配置变更，重启后即可用自然语言让智能体执行逆向分析。若只想在终端试用，可运行 `npx -y rea-agents@latest analyze-javascript-application <路径> --json` 分析一个 Electron 应用。贡献方面，建议先阅读 `docs/mcp-contracts.md` 了解 MCP 工具契约，再从文档完善或新增分析器适配入手。

#2

C++

连续 5 天

AnyPS5

Tool for automatic PS5 executables porting to Linux and Windows

Stars **20,357** Forks **1,628** Today **+5,925**

<https://github.com/boykopovar/AnyPS5>

项目简介：AnyPS5 是一个将 PS5 可执行文件自动移植到 Linux 和 Windows 平台的开源工具。它通过重链接器将 PS5 二进制转换为目标系统的原生格式，并实现系统 PRX 库以支持动态链接，从而让 PS5 游戏无需模拟器即可在 PC 上原生运行。

核心功能：一是重链接器（relinker），将 PS5 可执行文件转换为目标平台原生格式；二是系统 PRX 库实现，提供适合动态链接的系统库替代；三是着色器重编译器，可将 PS5 着色器转换为 SPIR-V；四是 SDL 手柄及键鼠输入映射支持；五是游戏兼容性追踪与验证列表。

适用场景：主要面向游戏保存与互操作性研究者、希望在现代 PC 硬件上运行 PS5 独占作品的玩家，以及对主机二进制移植技术感兴趣的系统级开发者。目前已有如《Dreaming Sarah》等 2D 游戏在 GTX 1050 Ti 上稳定 60fps 运行的验证案例。

技术亮点：项目采用原生动态链接而非模拟或独立运行时进程，这是一个根本性的架构选择；着色器重编译器能生成经 Spirv-Tools 验证的 SPIR-V 代码；遇到不支持状态时严格抛出 `std::runtime_error` 并终止进程，保证行为可预测；使用 C++ 编写，遵循 GPLv2 许可证，且明确声明不包含任何版权软件、固件或密钥。

上手建议：用户可先查阅 `docs/user/USAGE.md` 了解使用方法，并参考兼容性列表确认目标游戏是否受支持；开发者应从 `docs/dev/BUILD.md` 获取构建指引，通过 `docs/dev/ARCHITECTURE.md` 理解整体架构，并阅读技术债务文档和代码风格约定后再参与贡献。

#3

Shell

连续 4 天

skills

Skills for Real Engineers. Straight from my .agents directory.

Stars **282,287** Forks **23,649** Today **+1,696**

<https://github.com/mattpocock/skills>

项目简介：这是 TypeScript 教育者 Matt Pocock 开源的 AI Agent「技能包」集合，直接来自他个人的 .agents 目录。它针对当前 AI 编码代理（如 Claude Code、Codex）常见的两大痛点——「代理没做我想要的」和「代理输出太啰嗦」——提供了一套轻量、可组合的解决方案，强调「真正的工程」而非「vibe coding」。

核心功能：最核心的是 /grill-me 和 /grill-with-docs 两个「拷问」技能，让 Agent 在动手前反过来追问你的真实需求，从源头解决人机对齐问题；其次是 /triage（用标签管理 issue 分流）和 /setup-matt-pocock-skills（一次性配置 issue tracker、标签、文档路径）；整套技能按 engineering / productivity 分类，支持按需安装组合。

适用场景：适合日常使用 Claude Code、Codex、Copilot、Gemini CLI 等编码代理的工程师，尤其是在做真实项目（而非玩具 demo）时，希望 Agent 更懂自己的意图、输出更克制。也适合想自定义 Agent 工作流、把工程经验沉淀成可复用「技能」的开发者。

技术亮点：一是设计哲学明确——刻意做得「小、易改、可组合」，不接管你的流程（对比 GSD、BMAD、Spec-Kit 这类重型框架），因此出 bug 时容易定位修复；二是模型无关，任何 Agent 都能用；三是分发方式紧跟生态，同时支持插件市场（自动更新）和 npx skills 手动拷贝（可编辑），兼顾便利与可控。

上手建议：最快路径是选一个 Agent，用 README 里对应的插件命令安装（如 Claude Code 一条命令），然后跑一次 /setup-matt-pocock-skills 完成仓库级配置，之后每次改动前先调用 /grill-me 或 /grill-with-docs。想深入或贡献的话，直接从 skills/engineering 和 skills/productivity 目录读 SKILL.md 源码入手，fork 后按自己的习惯改造——作者本人也鼓励「hack around with them, make them your own」。

#4

HTML

连续 3 天

diagram-design

Editorial diagram design for Claude Code, Codex, GitHub Copilot, Factory Droid, and Pi. 42 diagram types. Self-contained HTML + SVG. No shadows. No Mermaid slop.

Stars **47,585** Forks **3,017** Today **+1,744**

<https://github.com/cathrynlaavery/diagram-design>

项目简介：这是一个为 AI 编程助手（Claude Code、Codex、GitHub Copilot、Factory Droid、Pi 等）设计的"编辑级图表设计"技能包，作者因厌倦 AI 生成的通用圆角框图而自建。它让 AI 直接产出符合品牌调性的高质量图表，而非千篇一律的 Mermaid 风格。

核心功能：提供 42 种图表类型（架构图、流程图、时序图、ER 图、象限图、Sankey、Wardley map、UML 类图等），每种都有浅色、深色、全编辑三种静态变体；通过读取你的网站自动匹配品牌配色，60 秒完成定制；还能把 draw.io、Mermaid、Excalidraw 的源文件重绘成指定格式和精度。

适用场景：适合写技术博客、做产品文档、画架构草图的内容创作者和工程师，尤其是那些既不想花 30 分钟在 Figma 里调色、又对 AI 默认输出审美不满的人。典型场景是"我需要一张配图，但不想为它开一个设计工具"。

技术亮点：输出是自包含的 HTML + SVG，无需构建步骤、无 JavaScript、无外部图片依赖，直接浏览器打开即可；采用"语义模式与布局分离"的设计，让队列、策略追踪、信任边界等行为描述复用现有图表类型而不膨胀类型数量；默认静态输出，可选动效仅用于有序讲解。

上手建议：先访问项目官网 diagramdesign.dev 看效果，再把它作为 Skill 装进你常用的 AI 编程助手（Claude Code / Codex 等），从最简单的架构图或流程图开始试用。想贡献的话，可以关注 README 提到的布局语法扩展方向——2.5.10 刚加了 10 种新布局，这类增量补充应该会比较友好的切入点。

#5

Go

NEW

open-code-review

Secure, fast, efficient, battle-tested at Alibaba's scale. Hybrid architecture code review tool: deterministic pipelines + LLM Agent, precise line-level comments, built-in multi-language ruleset (NPE, thread-safety, XSS, SQL injection), OpenAI & Anthropic compatible.

Stars **44,974** Forks **3,253** Today **+323**

<https://github.com/alibaba/open-code-review>

项目简介：Open Code Review 是阿里巴巴开源的一款 AI 驱动的代码审查 CLI 工具，源自阿里内部使用两年、服务数万名开发者的官方 AI 代码审查助手。它通过读取 Git diff 并调用可配置的大模型，自动生成行级精确的审查意见，解决人工代码审查耗时、易漏、标准不统一的问题。

核心功能：一是基于 Agent 的深度审查，模型可读取完整文件、检索代码库、查看其他变更文件以获取上下文，而非只看 diff 表面；二是内置多语言规则集，覆盖 NPE、线程安全、XSS、SQL 注入等典型缺陷；三是 ocr scan 支持对无 diff 的整文件或整个目录进行审查，便于审计陌生代码库；四是输出行级精确定位的结构化评论；五是兼容 OpenAI 与 Anthropic 接口，支持 Claude Code、Codex、Cursor、Kimi Code 等多种 Agent。

适用场景：适合中大型研发团队在 CI/CD 流程或本地提交前自动做代码审查，尤其适合需要统一安全与质量标准的工程团队；也适合个人开发者审查自己或开源项目的 PR，以及需要快速理解陌生代码库的审计场景。

技术亮点：采用"确定性流水线 + LLM Agent"的混合架构，在保证精度的同时大幅降低 token 消耗——官方基准显示，相同底层模型下其 Precision 和 F1 显著高于通用 Agent，token 消耗仅约 1/9，速度更快。项目还特意在 Recall 上做出取舍以换取低噪声，并基于 50 个仓库、200 个真实 PR、10 种语言、1505 条人工标注问题构建了 AACR-Bench 基准，工程严谨度较高。

上手建议：最简单的方式是通过 npm 安装 @alibaba-group/open-code-review，配置好模型 endpoint 后即可对当前仓库运行审查；想深入了解可访问官网 open-codereview.ai 和 DeepWiki。若想贡献代码，建议先阅读仓库的架构文档与规则集实现，从新增语言规则或优化 Agent 提示词等切入口较小的方向入手。

#6

Python

连续 2 天

knowledge-work-plugins

Open source repository of plugins primarily intended for knowledge workers to use in Claude Cowork

Stars **28,084** Forks **3,237** Today **+714**

<https://github.com/anthropics/knowledge-work-plugins>

项目简介：这是 Anthropic 官方开源的一套 Claude 插件集合，目标是把 Claude 从通用助手变成特定职能岗位的"专家同事"。它解决的核心问题是：知识工作者每天面对的工具和数据源极其分散，而通用 AI 不知道你公司的术语、流程和偏好，插件机制让 Claude 能按角色、按团队来定制工作方式。

核心功能：一是覆盖 11 个职能领域的开箱即用插件（销售、法务、财务、市场、产品、客服、数据分析、生物研究等），每个都内置了该岗位领域知识和最佳实践；二是通过 MCP 协议连接 Slack、Notion、Jira、HubSpot、Snowflake 等主流工具，让 Claude 能真正读写你的业务数据；三是提供 slash 命令（如 /sales:call-prep、/data:write-query）作为显式触发的工作流入口；四是支持企业自定义，团队可以把插件改造成符合自己工具链和术语的版本。

适用场景：主要面向企业中的知识工作者——销售做客户调研和通话准备、法务审合同和分流 NDA、财务做对账和报表、产品经理写 spec 和整理用户研究等。另一个重要场景是团队协作层面：当整个团队安装同一套定制插件后，输出质量和一致性会显著提升，而不是每个人各自"调教"AI。

技术亮点：插件采用统一的文件化结构（plugin.json 清单 + commands/ + skills/ + .mcp.json），其中 Skills 是自动触发的领域知识，Commands 是用户显式调用的动作，Connectors 通过 MCP 服务器对接外部工具——这种"自动 vs 显式"的分层设计值得关注。同时它兼容 Claude Cowork 和 Claude Code 两个产品，并通过 marketplace 机制分发，说明 Anthropic 在把插件当作一个可扩展的生态平台来建设。

上手建议：最简路径是在 Cowork 里从 claude.com/plugins 直接安装，或在 Claude Code 中用 `claude plugin marketplace add anthropics/knowledge-work-plugins` 添加市场后安装具体插件（如 `sales@knowledge-work-plugins`）。想深入的话，建议先挑一个和自己岗位最接近的插件读它的 `skills/` 目录，理解领域知识是怎么组织的；想做贡献或定制，参考 `cowork-plugin-management` 插件来创建自己的插件。

#7

Python

NEW

litellm

The fastest, lightest AI Gateway. Rust core with Python SDK. Call 100+ LLM APIs in OpenAI (or native) format with cost tracking, guardrails, load balancing, and logging [Bedrock, Azure, OpenAI, Anthropic, OpenAI, VertexAI, vLLM, Nvidia NIM]

Stars **60,529** Forks **12,177** Today **+95**

<https://github.com/BerriAI/litellm>

项目简介

LiteLLM 是一个开源 AI 网关，为 100+ LLM 提供商（OpenAI、Anthropic、Gemini、Bedrock、Azure 等）提供统一的 OpenAI 格式调用接口。它解决的核心问题是：不同 LLM 提供商的 SDK、认证方式、请求格式和错误类型各不相同，开发者需要为每个提供商单独适配，维护成本极高。LiteLLM 通过一层抽象抹平这些差异，让开发者用同一套代码调用任意模型。

核心功能

- **统一 API 接口**：用 OpenAI 格式调用 100+ LLM，无需切换各厂商 SDK，实现"一次编写，随处调用"。
- **AI Gateway（代理服务器）**：可自托管部署为集中式网关，提供虚拟密钥管理、花费追踪、负载均衡、护栏（guardrails）和日志记录。
- **成本追踪与预算控制**：内置消费统计和预算限制，适合团队或企业级成本治理。
- **高吞吐低延迟**：Rust 核心 + Python SDK，官方基准测试显示 1k RPS 下 P95 延迟仅 8ms。
- **管理后台**：开箱即用的 Admin Dashboard，方便运维和监控。

适用场景

- **多模型应用开发者**：需要在同一应用中灵活切换或组合多个 LLM 提供商，避免供应商锁定。
- **企业 AI 平台团队**：需要集中管理 API 密钥、追踪各部门花费、设置预算上限和访问控制。
- **Agent / 框架开发者**：作为底层模型调用层，统一处理不同提供商的请求与错误（Stripe、Google ADK、OpenHands 等已采用）。

技术亮点

- **Rust 核心 + Python SDK 双栈架构**：兼顾性能（网关层用 Rust 处理高并发）与生态兼容性（Python 是 AI 领域主流语言）。
- **OpenAI 格式作为"通用语言"**：利用 OpenAI API 事实上的行业标准地位，降低迁移和适配成本。
- **生产级特性内建**：虚拟密钥、负载均衡、护栏、日志等通常需要自行搭建的能力直接内置，减少重复造轮子。

上手建议

- **快速体验**：通过 Render / Railway / AWS / GCP 的一键部署按钮启动 Proxy Server，或 `pip install litellm` 直接作为 SDK 使用。
- **深入文档**：优先阅读 [LiteLLM Proxy 文档](#) 和 [支持的提供商列表](#)。
- **参与贡献**：项目由 YC W23 团队 BerriAI 维护，活跃于 Discord/Slack，可从新增 Provider 适配或文档改进入手，是门槛较低的贡献切入点。

#8

JavaScript

NEW

agent-skills

Production-grade engineering skills for AI coding agents.

Stars **103,777** Forks **10,852** Today **+523**

<https://github.com/addyosmani/agent-skills>

项目简介

这是 Google 工程师 Addy Osmani 推出的「AI 编码代理技能库」，把资深工程师在软件开发全流程中使用的工作流、质量门禁和最佳实践打包成可复用的技能（Skills），让 AI 代理在每个开发阶段都能稳定遵循同一套标准。它解决的核心问题是：AI 写代码往往「能跑就行」，缺乏工程规范，而该项目把「资深工程师的做事方式」显式注入到代理行为中。

核心功能

- **9 个斜杠命令映射开发全生命周期**：从 `/spec`（先写规格再写码）、`/plan`（拆成原子任务）、`/build`、`/test`、`/review` 到 `/ship`，每个命令自动激活对应技能。
- **25 个可独立安装的技能**：如 `code-review-and-quality`（合并前的五维审查）、`test-driven-development`（强制红-绿-重构）、`interview-me`（一次一问的需求拷问）。
- **自动触发机制**：设计 API 时自动激活 `api-and-interface-design`，写 UI 时激活 `frontend-ui-engineering`，无需手动调用。
- **`/build auto` 自主模式**：批准一次计划后自动完成全部任务，但每个任务仍走测试驱动并单独提交，遇失败或高风险步骤会暂停。
- **跨工具一键安装**：通过 `npx skills` CLI 可安装到 70+ 代理（Claude Code、Cursor、Codex、Copilot、Cline 等），也支持 Claude Code 原生插件市场。

适用场景

适合使用 AI 编码代理进行实际项目开发的工程师和团队——尤其是希望 AI 产出达到生产级质量、而非仅能演示的场景。典型用法是在一个新功能或新项目开始时，用 `/spec` 理清需求、`/plan` 拆解任务，再让代理按技能规范逐步构建、测试、审查和发布。也适合个人开发者把资深工程师的「隐性经验」变成可复用的显性流程。

技术亮点

- **「技能」作为工程知识的封装单元**：把 workflow、质量门禁、最佳实践做成模块化、可组合、可自动激活的资产，而非零散的提示词。
- **人机协作边界设计巧妙**：`/build auto` 移除的是「任务之间的人工干预」，而非「验证环节」——每个任务依然测试驱动、独立提交、失败即停，兼顾效率与安全。
- **高度可移植**：基于开放的 skills CLI 标准，一套技能可跨 70+ 代理使用，避免绑定单一工具。
- **诚实暴露局限**：README 明确指出单技能安装不会复制仓库级 `references/` 目录，并附上 issue 追踪，工程态度务实。

上手建议

最快路径是运行 `npx skills add addyosmani/agent-skills --list` 先浏览全部技能，再按需安装；Claude Code 用户推荐用插件市场方式 (`/plugin marketplace add addyosmani/agent-skills`) 获得完整集成。想贡献的话，建议先从一个具体技能目录 (`skills/<name>/`) 入手，理解其结构后再参考现有技能补充或改进，并注意仓库级 `references/` 的共享依赖关系，避免只改单技能而遗漏配套清单。

#9

Rust

连续 2 天

artcraft

ArtCraft is an intentional crafting engine for artists, designers, and filmmakers

Stars **10,516** Forks **1,570** Today **+3,723**

<https://github.com/storytold/artcraft>

项目简介：ArtCraft 是一款面向艺术家、设计师和电影制作人的“AI 创作 IDE”，用 Rust 构建。它试图解决当前 AI 图像/视频生成中“提示词不可控、结果不可复现”的核心痛点，把随机的“提示（prompting）”变成可精确操控的“手艺（crafting）”。

核心功能：一是 2D/3D 场景合成，支持图层叠加、背景去除、3D 道具布景；二是 Image to Location，让虚拟角色在一致环境中反复出镜、规划多镜头；三是 Image to 3D Mesh 与角色摆姿，可在生成前精确设定相机角度和人物姿态；四是 Kitbashing 场景搭建，用 3D 资产套件控制构图与景深。

适用场景：适合需要稳定角色形象和镜头连续性的创作者，比如分镜设计、漫画/动画预演、短视频与广告拍摄、概念艺术。典型用法是先在 3D 空间里“搭好场景”，再调用不同 AI 模型完成最终渲染，而不是反复抽卡式地试提示词。

技术亮点：用 Rust 实现，意味着对实时 3D 渲染、图形管线和性能敏感操作有天然优势；同时它把“模型选择权”交给用户，不绑定单一 AI 后端，定位更接近中立的创作层。将 2D 合成、3D 场景、网格生成与角色系统整合进一个 IDE，是它区别于普通 AI 绘图工具的关键。

上手建议：普通用户可直接从官网 getartcraft.com 下载，或看两分钟的 YouTube 演示快速理解 workflow；开发者可从 GitHub Releases 获取版本，参照 `_docs/dev_setup.md` 从源码构建。想贡献的话，建议先加入 Discord 社区、跑通本地构建，再从 issue 和功能讨论入手。

#10

Python

NEW

lingbot-map

[ECCV 2026 Best Paper Award Candidate] LingBot-Map: Geometric Context Transformer for Streaming 3D Reconstruction

Stars **17,604** Forks **1,945** Today **+109**

<https://github.com/Robbyant/lingbot-map>

项目简介：LingBot-Map 是 Robbyant 团队开发的一个前馈式（feed-forward）3D 基础模型，专注于流式 3D 重建任务。它试图解决传统方法在长序列视频中实时重建 3D 场景时面临的漂移累积和推理效率低下的问题，让模型能够像"边看边建图"一样持续处理视频流。

核心功能：一是**几何上下文 Transformer**，在单一流式框架内统一了坐标锚定、稠密几何线索和长程漂移校正；二是**高效流式推理**，借助分页 KV 缓存注意力机制，在 518×378 分辨率下可对超过 1 万帧的长序列保持约 20 FPS 的稳定推理；三是**离线渲染管线**，支持超长视频（如 2.5 万帧、13 分钟的室内漫游）的批量重建与渲染；四是提供覆盖 KITTI、Oxford Spires、TUM-D 等 9 个数据集的完整评测基准。

适用场景：主要面向做 SLAM、三维重建、机器人导航、AR/VR 空间理解的研究者和工程师。典型使用场景包括：从手机或机器人拍摄的长视频中实时重建场景几何、自动驾驶中的在线建图、以及需要处理超长视频序列的离线三维内容生产。

技术亮点：架构上通过"锚点上下文 + 位姿参考窗口 + 轨迹记忆"三种机制实现长程漂移校正，这是它区别于普通流式重建方法的关键。工程上采用分页 KV 缓存和 FlashInfer 后端，把 Transformer 的显存瓶颈转化为可管理的分页调度，从而支撑万帧级序列。此外，项目同时提供会议版和技术报告版论文，并已开源多数据集评测脚本，学术透明度较高。

上手建议：建议先用 conda 创建 Python 3.10 环境，安装推荐的 PyTorch 2.8.0 + CUDA 12.8，然后从 demo.py 入手跑通示例场景，再尝试关键帧间隔和窗口化推理（针对超过 3000 帧的长序列）。如果要做离线渲染，需额外安装 NVIDIA Kaolin。想贡献代码的话，可以从 TODO 列表中已完成的评测基准出发，补充新的数据集支持或优化 SDPA/FlashInfer 后端的推理性能。

数据来源: github.com/trending

报告日期: 2026-10-09

由 DeepSeek AI 提供智能分析 | 自动生成